

✓ 1INF24 Inteligencia Artificial

Laboratorio 3 - Parte 1: Pre-procesamiento de datos

Indicaciones generales:

- Las respuestas deben contar con **fundamento teórico**.
- Cualquier indicio de plagio resultará en la anulación de la prueba.
- Se espera que sus respuestas se justifiquen con ayuda de los resultados de ejecución en los casos que se solicite. **No se calificarán aquellas respuestas que no presenten un resultado de ejecución o que no concuerden con este.**
- Subir el cuadernillo con el nombre **Lab03_código.ipynb**, donde código es su código PUCP de 8 dígitos.
- Está prohibido el uso de herramientas de IA. En caso se detecte el uso de estas herramientas la calificación será 00.
- Solo se permite el cheat sheet de pandas y los notebooks de pre-procesamiento dados en clase

Caso:

Este laboratorio se basa en un conjunto de datos de juegos de mesa hasta 2025 obtenido de BoardGameGeek. El dataset a utilizar contiene información detallada sobre miles de juegos de mesa y sus características. Las columnas más relevantes incluyen:

Columna	Descripción
name	Nombre del juego de mesa (categórica).
yearpublished	Año de publicación del juego (numérica).
minplayers	Número mínimo de jugadores recomendado (numérica).
maxplayers	Número máximo de jugadores recomendado (numérica).
playingtime	Tiempo de juego promedio en minutos (numérica).
minplaytime	Tiempo mínimo de juego en minutos (numérica).
maxplaytime	Tiempo máximo de juego en minutos (numérica).
minage	Edad mínima recomendada (numérica).
boardgamecategory	Categorías del juego (categórica). Ejemplo: 'estrategia', 'familia', 'cartas', etc. Como arreglo de cadenas.
boardgamemechanic	Mecánicas de juego (categórica). Ejemplo: 'gestión de recursos', 'construcción de mazos', etc. Como arreglo de cadenas.
boardgamedesigner	Diseñadores del juego (categórica). Como arreglo de cadenas.
boardgamepublisher	Editoriales que han publicado el juego (categórica). Como arreglo de cadenas.
average	Puntuación media asignada por los usuarios (numérica: 1-10).
bayesaverage	Puntuación media bayesiana (ajustada por número de votos) (numérica: 1-10).

Columna	Descripción
Board Game Rank	Posición en el ranking general de juegos de mesa.
usersrated	Número de usuarios que han valorado el juego (numérica).
owned	Número de usuarios que indican poseer el juego (numérica).
averageweight	Complejidad media del juego según los usuarios (numérica: 1-5, donde 5 es más complej)
suggested_num_players	Número de jugadores sugerido basado en valoraciones de usuarios (array de diccionario
suggested_playerage	Edad de juego sugerida basada en valoraciones de usuarios (array de diccionarios).

El objetivo de este laboratorio será aplicar técnicas de preprocesamiento de datos sobre este conjunto para garantizar la calidad y consistencia de los datos antes de su uso en otros modelos.

✓ Descarga de datos

```
# No modificar

# El dataset puede ser descargado mediante esta celda o desde PAIDEIA

import requests

response = requests.get(
    "https://www.kaggle.com/api/v1/datasets/download/jvanelteren/boardgamegeek-
    allow_redirects=True,
    stream=True
)
response.raise_for_status()
with open('games_detailed_info2025.csv', 'wb') as file:
    for chunk in response.iter_content(chunk_size=8192):
        file.write(chunk)

print("Archivo descargado exitosamente")
```

✓ Parte 1

```
# No modificar

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

warnings.simplefilter(action='ignore', category=FutureWarning)
sns.set(style="whitegrid")
pd.set_option('display.float_format', lambda x: '%.2f' % x)
```

✓ Pregunta 1 (1 punto)

Leer correctamente el archivo. Considere utiliza la columna 'id' como índice. Elimine la primera columna sin nombre. Elimine estas columnas que no serán necesarias para este laboratorio:

```
columns=['thumbnail', 'alternate', 'boardgamefamily', 'boardgameexpansion',
'boardgameartist', 'boardgameimplementation', 'boardgameaccessory',
'boardgamecompilation', 'boardgamepublisher', 'boardgameintegration', 'RPG Item Rank',
'Accessory Rank']
```

Muestre las dimensiones del DataFrame.

```
# Completar
df =
```

✓ Pregunta 2 (2 puntos)

Analice las columnas yearpublished, minplayers, averageweight y average. Por cada una, muestre (con un gráfico de barras, histograma, etc) si hay datos incorrectos de acuerdo a lo descrito en la tabla de arriba. Considere además los siguientes criterios:

- yearpublished: Debe ser un número entero entre 1900 y 2025, inclusive.
- minplayers: Debe ser un número entero entre 1 y 20, inclusive.

Responda en la celda de texto cuáles son las columnas con datos incorrectos y qué datos son incorrectos.

Tip: Puede usar seaborn con `sns.histplot()` o `sns.countplot()` para graficar.

```
# Completar. Puede usar estas funciones de seaborn u otras que prefiera
sns.histplot()
sns.countplot()
```

Responder aquí:

▼ Pregunta 3 (3 puntos)

- De acuerdo a lo respondido en la pregunta anterior, elimine las filas que contengan datos incorrectos. Muestre las dimensiones del DataFrame después de la limpieza. ¿Es siempre correcto eliminar filas con algunos datos incorrectos? Justifique su respuesta.
- Además, analice (con describe() o algún gráfico) la columna 'median' y 'type'. ¿Qué recomienda hacer con ellas? Realice la acción recomendada.
- La columna 'usersrated' contiene el número de usuarios que han valorado el juego. Para este laboratorio, considere que un juego con menos de 50 valoraciones no es relevante. Elimine los juegos que no cumplan con este criterio. Muestre las dimensiones del DataFrame después de la limpieza.

```
# Completar
print("Cantidad de filas antes de eliminar:", df.shape[0])
df=df[?]
```

```
# Completar
```

```
# Completar
print("Cantidad de filas antes de eliminar:", df.shape[0])
df = df[?]
```

▼ Pregunta 4 (4 ptos)

Muestre la cantidad de valores nulos en las columnas que tienen Rank en el nombre. Considerando que estas columnas son indicadores de la posición en el Ranking de determinada categoría (ej. tener 10 en Party Game Rank significa que el juego es el décimo mejor juego de mesa en la categoría Party Game), ¿qué recomienda hacer con estos valores nulos? Realice la acción recomendada y justifique su respuesta.

Complete los valores faltantes de las demás columnas con la media, moda o mediana según su criterio. Justifique sus elecciones

```
# Completar
columnasConRank = df.columns[df.columns.str.contains('Rank')].tolist()

for col in columnasConRank:
```

Responder aquí:

```
#Completar valores faltantes de las demás columnas aquí

# Verificación rápida
print("Nulls tras imputación:")
print(df[['suggested_playerage', 'suggested_language_dependence']].isnull().sum()
```

▼ Pregunta 5 (2 ptos)

Se le proporciona el esqueleto de la función `diccionario_a_unico_valor` que transforma una columna de un DataFrame que contiene diccionarios en forma de string a un único valor. Complete la función en las partes donde se indique `#COMPLETAR`. Se busca que la función devuelva el valor más sugerido por los usuarios. Para el caso de `suggested_num_players`, se debe devolver el número de jugadores marcado como Best más veces.

Ejemplo de funcionamiento:

Ejemplo antes de procesar:

```
[{'@level': '1', '@value': 'No necessary in-game text', '@numvotes': '21'}, {'@level': '2', '@value': 'Some necessary text - easily memorized or small crib sheet', '@numvotes': '8'}, {'@level': '3', '@value': '4', '@numvotes': '4'}]

[{'@value': '2', '@numvotes': '1'}, {'@value': '3', '@numvotes': '0'}, {'@value': '4', '@numvotes': '4'}]

[{'@numplayers': '1', 'result': [{'@value': 'Best', '@numvotes': '1'}, {'@value': '4', '@numvotes': '4'}]}, {'@numplayers': '2', 'result': [{'@value': 'Best', '@numvotes': '1'}, {'@value': '4', '@numvotes': '4'}]}, {'@numplayers': '3', 'result': [{'@value': 'Best', '@numvotes': '1'}, {'@value': '4', '@numvotes': '4'}]}]
```

Ejemplo luego de procesar:

```
Some necessary text - easily memorized or small crib sheet (se devuelve el valor más votado)
8 (se devuelve el valor más votado)
4 (se devuelve el valor donde Best tiene más votos)
```

Luego responda: ¿Qué tipo de dato se obtiene? ¿Son siempre enteros? ¿Será necesaria alguna otra conversión?

```
import json
import ast

def procesar_columna_sugerencias(df, columna, tipo_extraccion='mejor_valor'):
    """
    Procesa una columna que contiene datos JSON y retorna el valor más relevante.

    Parámetros:
    """
```

```

df (DataFrame): DataFrame que contiene la columna a procesar
columna (str): Nombre de la columna a procesar
tipo_extraccion (str): 'mejor_valor' o 'mejor_jugadores'

Retorna:
Series: La columna procesada con los valores extraídos
"""
def parse_json_string(json_str):
    if pd.isna(json_str):
        return None
    try:
        return json.loads(json_str) if isinstance(json_str, str) else json_
    except:
        try:
            return ast.literal_eval(json_str) if isinstance(json_str, str)
        except:
            return None

def procesar_item(json_data):
    if not json_data:
        return np.nan

    # Para suggested_num_players (extracción de mejor_jugadores)
    if tipo_extraccion == 'mejor_jugadores':
        mejor = {'numplayers': np.nan, 'votos': 0}
        for item in json_data:
            if '@numplayers' in item and 'result' in item:
                for resultado in item['result']:
                    if resultado['@value'] == 'Best' and int(resultado['@nu
                        #Completar
        return mejor['numplayers']

    # Para suggested_playerage y suggested_language_dependence (mejor_valor
    mejor = {'valor': np.nan, 'votos': 0}
    for item in json_data:
        # Estructura de suggested_playerage
        if '@value' in item and '@numvotes' in item:
            votos = #Completar
            if votos > mejor['votos']:
                #Completar
        # Estructura de suggested_language_dependence
        elif '@level' in item and '@value' in item and '@numvotes' in item:
            votos = #Completar
            #Completar

    return mejor['valor']

# Aplicar procesamiento a la columna
return df[columna].apply(parse_json_string).apply(procesar_item)

```

```
print("Ejemplo antes de procesar:")
print(df['suggested_language_dependence'][13])
print(df['suggested_playerage'][13])
print(df['suggested_num_players'][13])
```

```
# Ejemplo de uso:
df['suggested_playerage'] = procesar_columna_sugerencias(df, 'suggested_playerage')
df['suggested_num_players'] = procesar_columna_sugerencias(df, 'suggested_num_players')
df['suggested_language_dependence'] = procesar_columna_sugerencias(df, 'suggested_language_dependence')
```

```
print("Ejemplo luego de procesar:")
print(df['suggested_language_dependence'][13])
print(df['suggested_playerage'][13])
print(df['suggested_num_players'][13])
```

```
print("Tipo de datos de suggested_playerage:", df['suggested_playerage'].dtype)
```

Responder aquí:

Para verificar la transformación hasta ahora, se le proporciona la función `mostrar_juego`. Se ejecuta para el juego en la posición 1 del ranking de juegos de fiesta. Debería mostrar el juego "Blood on the Clocktower". Con 12 jugadores recomendados y una edad sugerida de 12 años.

```
#No modificar
def mostrar_juego(df, id_juego):
    """
    Muestra información detallada de un juego de mesa con formato mejorado, incluyendo imágenes.

    Parámetros:
    df (DataFrame): DataFrame que contiene los juegos
    id_juego (int): ID del juego a mostrar
    """
    import matplotlib.pyplot as plt
    import matplotlib.font_manager as fm
    import requests
    from PIL import Image
    from io import BytesIO
    import numpy as np
    import platform

    # Verificar si el juego existe en el DataFrame
    if id_juego not in df.index:
        print(f"⚠ El juego con ID {id_juego} no se encuentra en el DataFrame.")
        return

    juego = df.loc[id_juego]
```

```

# Configurar fuente que soporte emojis según el sistema operativo
if platform.system() == 'Windows':
    emoji_font = 'Segoe UI Emoji' # Fuente con emojis en Windows
elif platform.system() == 'Darwin': # macOS
    emoji_font = 'Apple Color Emoji'
else: # Linux y otros
    emoji_font = 'Noto Color Emoji'

# Verificar si la fuente está disponible
font_available = any(f for f in fm.findSystemFonts() if emoji_font.lower()

# Configurar fuente para el gráfico
if font_available:
    plt.rcParams['font.family'] = emoji_font

# Obtener categorías y mecánicas si están disponibles
categorias = []
mecanicas = []

try:
    if not pd.isna(juego['boardgamecategory']):
        cat_str = juego['boardgamecategory'].replace('[', '').replace(']',
        categorias = [c.strip() for c in cat_str.split(',') if c.strip()]
except:
    pass

try:
    if not pd.isna(juego['boardgamemechanic']):
        mec_str = juego['boardgamemechanic'].replace('[', '').replace(']',
        mecanicas = [m.strip() for m in mec_str.split(',') if m.strip()]
except:
    pass

# Crear figura y ejes directamente
fig, (ax_img, ax_text) = plt.subplots(1, 2, figsize=(12, 7),
                                     gridspec_kw={'width_ratios': [1, 2]},

# Configurar áreas
ax_img.axis('off')
ax_text.axis('off')

# Intentar cargar y mostrar la imagen
try:
    if 'image' in df.columns and not pd.isna(juego['image']):
        response = requests.get(juego['image'])
        img = Image.open(BytesIO(response.content))
        ax_img.imshow(img)
        ax_img.set_title("Portada del juego", fontsize=12, pad=10)
    else:
        # Mostrar un placeholder si no hay imagen
        ax_img.text(0.5, 0.5, "Imagen no disponible",
                    horizontalalignment='center', verticalalignment='center'
                    fontsize=12, color='gray')
except:
    ax_img.text(0.5, 0.5, "Error al cargar la imagen",

```

```

        horizontalalignment='center', verticalalignment='center',
        fontsize=12, color='gray')

# Usar símbolos de texto simples si hay problemas con los emojis
info_text = (
    f"JUEGO: {juego['name']}\n\n"
    f"ID: {juego.name}\n"
    f"Año: {int(juego['yearpublished'])}\n"
    f"Jugadores: {int(juego['minplayers'])} - {int(juego['maxplayers'])}\n"
    f"Tiempo: {int(juego['playingtime'])} minutos\n"
    f"Complejidad: {juego['averageweight']:.2f}/5\n"
    f"Edad sugerida: {juego['suggested_playerage']} años\n"
    f"Jugadores sugeridos: {juego['suggested_num_players']} jugadores\n"
    f"Dependencia idioma: {juego['suggested_language_dependence']}\n\n"
    f"VALORACIONES:\n"
    f"• Puntuación: {juego['average']:.2f}/10\n"
    f"• Valoraciones: {int(juego['usersrated'])}\n\n"
    f"RANKINGS:\n"
)

# Añadir rankings disponibles
rankings = []
for col in df.columns:
    if 'Rank' in col and not pd.isna(juego[col]):
        rankings.append(f"• {col}: {int(juego[col])}")

if rankings:
    info_text += "\n".join(rankings)
else:
    info_text += "• Sin rankings disponibles"

# Añadir categorías
if categorias:
    info_text += "\n\nCATEGORÍAS:\n"
    for i, cat in enumerate(categorias[:4]):
        info_text += f"• {cat}\n"
    if len(categorias) > 4:
        info_text += f"• ... y {len(categorias)-4} más\n"

# Añadir mecánicas
if mecanicas:
    info_text += "\nMECÁNICAS:\n"
    for i, mec in enumerate(mecanicas[:4]):
        info_text += f"• {mec}\n"
    if len(mecanicas) > 4:
        info_text += f"• ... y {len(mecanicas)-4} más\n"

# Mostrar el texto en el área correspondiente
ax_text.text(0, 1, info_text, fontsize=11, va='top', ha='left', linespacing

# Título general
plt.suptitle(f"Ficha de Juego: {juego['name']}", fontsize=16, y=0.98)

# Ajustar layout
plt.subplots_adjust(left=0.05, right=0.95, top=0.90, bottom=0.05)

```

```
plt.show()
```

```
mostrar_juego(df, df.loc[df['Party Game Rank'] == 1].index[0]) #Ejemplo de uso,
```

✓ Pregunta 6 - Escalamiento (2 puntos)

Para el subconjunto de variables numéricas elegido (['yearpublished', 'usersrated', 'owned', 'average', 'averageweight']), debes:

- Mostrar la media y la desviación estándar antes de escalar.
- Aplicar MinMaxScaler y volver a mostrar esos mismos estadísticos.
- Aplicar StandardScaler y volver a mostrarlos por última vez.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# 1) subconjunto para analizar en la pregunta
subset = ['yearpublished', 'usersrated', 'owned', 'average', 'averageweight']

X = df[subset]
desc = X.describe()
print("Antes de escalar (subset):\n", desc.loc[['mean', 'std']])

# 2) MinMaxScaler sobre todas las numéricas
X_min = #Completar
df_min = #Completar
desc_min = df_min.describe()
print("\nDespués de MinMaxScaler (subset):\n", desc_min.loc[['mean', 'std']])

# 3) StandardScaler sobre todas las numéricas
X_std = # Completar
df_std = # Completar
desc_std = df_std.describe()
print("\nDespués de StandardScaler (subset):\n", desc_std.loc[['mean', 'std']])
```

¿Para qué sirve el escalado y estandarización de variables en el preprocesamiento de datos y qué sucede geométricamente con cada una de esas operaciones?

✓ Pregunta 7 - (2 puntos)

En el algoritmo Particle Swarm Optimization (PSO), ¿qué representan la fuerza inercial, el aprendizaje cognitivo y el aprendizaje social? Explica cómo cada uno influye en el

movimiento de las partículas y qué sucede si se incrementa o disminuye el valor del coeficiente de inercia (ω).

$$v_i^{k+1} = \omega \cdot v_i^k + \underbrace{\varphi_1 \cdot \text{rnd}_1 \cdot (pBest_i - x_i^k)}_{\text{Aprendizaje Cognitivo}} + \underbrace{\varphi_2 \cdot \text{rnd}_2 \cdot (g - x_i^k)}_{\text{Aprendizaje Social}}$$

RESPUESTA:

✓ Pregunta 8 (2 puntos)

Explica brevemente el rol de las abejas empleadas, observadoras y exploradoras en el algoritmo ABC. ¿Por qué es importante el mecanismo de abandono de soluciones (abejas scouts)?

RESPUESTA:

✓ Pregunta 9 - Codificación (2 puntos)

A continuación se describen dos variables categóricas:

Nivel de satisfacción del cliente: Bajo, Medio, Alto.

Color de producto: Rojo, Verde, Azul, Amarillo.

Para cada variable, indica qué técnica de codificación usarías — Label Encoding o One-Hot Encoding — y justifica brevemente tu elección.

RESPUESTA: